



武汉理工大学
WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发

项目：企业级 RAG 知识库

姓名：刘涛

学号：2025303004

班级：研 253

老师：戚欣

2026 年 6 月

目 录

一、选题背景与设计思想	1
1.1 问题定义	1
1.2 现有方案不足	1
1.3 项目价值	1
1.4 技术路线	1
1.5 核心技术要素	2
二、Specs 规格文档	2
2.1 Product Spec	2
2.2 Architecture Spec	4
2.3 API Spec	5
三、系统架构与设计	6
3.1 整体架构图	6
3.2 Agent 交互流程	6
3.3 数据流设计	6
3.4 技术选型理由	7
四、关键实现与代码展示	7
4.1 LangGraph Workflow 定义	7
4.2 RAGState 状态定义	8
4.3 Planner Agent 实现	8
4.4 混合检索实现	9
4.5 SSE 流式输出实现	10
4.6 AI IDE 辅助开发	11
五、测试与评估	13

5.1 单元测试.....	13
5.2 功能测试.....	13
5.3 性能指标.....	13
5.4 Demo 截图.....	14
六、系统升级与扩展.....	15
6.1 可扩展架构.....	15
6.2 下一阶段计划.....	15
6.3 能力演进路径.....	16
七、课程总结.....	16
7.1 个人收获.....	16
7.2 对课程的建议.....	16
7.3 总结.....	16
附录.....	16
A. 项目仓库.....	16
B. 环境变量配置.....	17
C. 快速启动.....	17

一、选题背景与设计思想

1.1 问题定义

企业内部有大量文档，像制度手册、技术文档、产品说明这些。员工想找点信息挺费劲的，主要几个问题：关键词搜索不太懂语义，得精确匹配才行；一个答案可能散落在好几个文档里，还得自己拼；没法多轮对话追问；答案对不对也没法验证，不知道出处是哪。

1.2 现有方案不足

传统 RAG 就是 retrieve 然后 answer，流程太简单了。只检索一次，复杂问题搞不定；检索结果不好也不知道，直接就生成答案了；遇到复合问题不会拆解；答案也没标注引用来源，可信度不高。

问题	表现	影响
单次检索	只执行一次检索，无法处理复杂问题	答案不完整
无质量判断	检索结果不佳时无法自适应	答案质量差
缺乏规划	无法拆解复杂问题	无法处理复合查询
无引用溯源	答案缺乏来源标注	可信度低

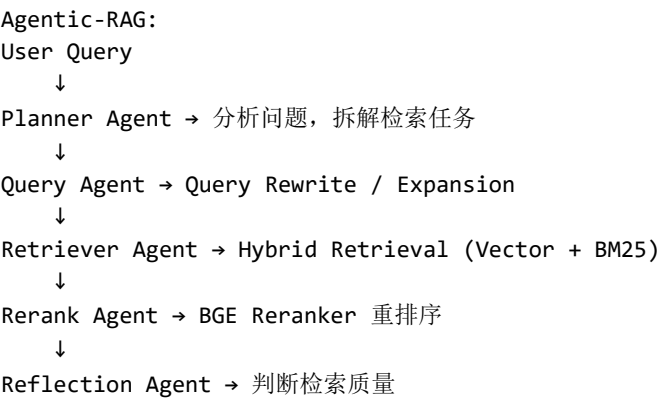
1.3 项目价值

Agentic-RAG 系统，主要是把传统 RAG 那种单一检索流程改成了多 Agent 协作的工作流。好处是能自动拆解问题、优化检索策略，检索结果不好还会重试，每个答案都标了引用来源，也能多轮对话。

1.4 技术路线

核心思路是把 retrieve → answer 改成多步：

传统 RAG: User Query → Retrieve → Answer



- └ pass → Answer Agent → 生成答案 + 引用来源
- └ fail → 重试检索 (max 2 次)

1.5 核心技术要素

技术要素	实现方式
SDD 规格驱动开发	Product Spec / Architecture Spec / API Spec
工具使用 / Function Calling	LangGraph Agent 节点调用检索、重排序等工具
记忆机制	Redis 存储对话历史，支持多轮上下文
状态管理与多步骤推理	LangGraph StateGraph 管理工作流状态
多智能体协作	6 个 Agent 分工协作完成检索问答
可观测性	结构化日志 + Retrieval Debug 可视化

二、Specs 规格文档

2.1 Product Spec

产品定位

给企业用的文档问答系统，基于 Agentic-RAG 技术做多阶段检索，能自适应优化，答案带引用。

目标用户

企业员工：查公司制度、流程、技术文档

知识管理员：管理企业文档库

系统管理员：看系统运行状态

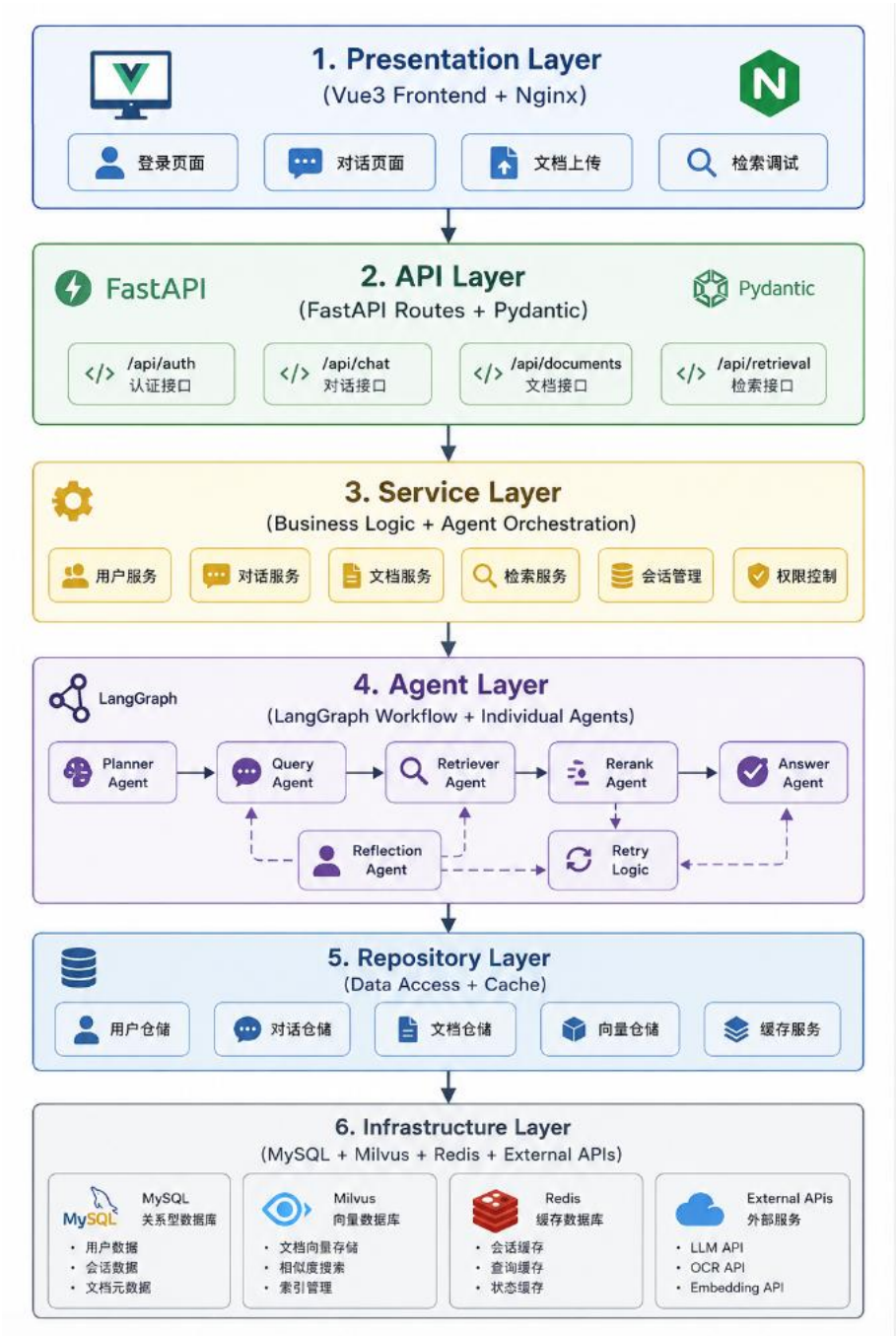
核心功能

功能模块	功能说明
文档管理	支持 PDF/DOCX/TXT/Markdown 上传、解析、切分、Embedding
智能问答	多阶段 Agent 工作流，支持多轮对话
引用溯源	每个答案标注文档来源，支持跳转查看
检索调试	可视化 Agent 执行流程和检索结果

用户故事

ID	用户故事	优先级
US-01	作为知识管理员，我想上传文档以便构建企业知识库	P0
US-02	作为企业员工，我想用自然语言提问以便快速获取信息	P0
US-03	作为企业员工，我想看到答案的引用来源以便验证准确性	P0
US-04	作为企业员工，我想进行多轮对话以便深入探讨问题	P1
US-05	作为系统管理员，我想查看检索调试信息以便优化系统	P2

2.2 Architecture Spec



数据库设计

系统包含 6 张核心数据表：

表名	说明	核心字段
users	用户表	id, username, email, password_hash

documents	文档表	id, user_id, filename, file_type, status
document_chunks	文档片段表	id, document_id, chunk_index, content
conversations	会话表	id, user_id, title
messages	消息表	id, conversation_id, role, content, citations_json
retrieval_logs	检索日志表	id, message_id, query, plan, retrieved_docs

缓存设计

数据类型	存储位置	TTL
对话历史	Redis List	24h
检索结果	Redis String	5min
会话元数据	Redis Hash	7d

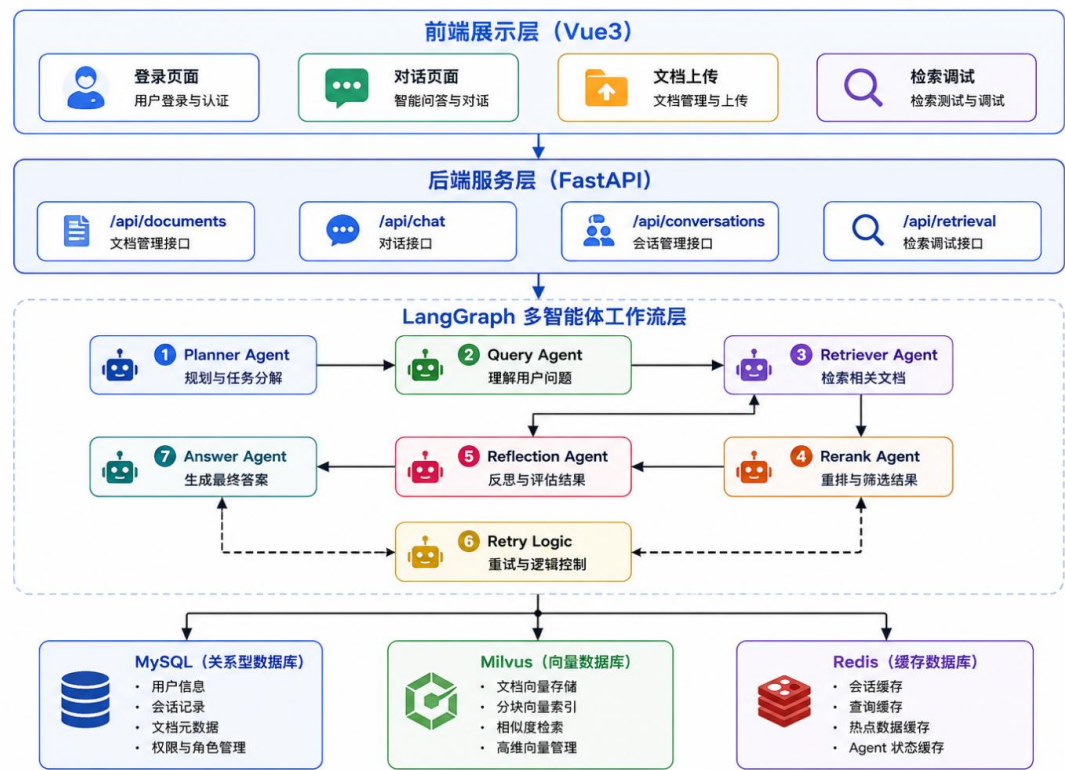
2.3 API Spec

核心接口

Method	Path	说明
POST	`/api/auth/register`	用户注册
POST	`/api/auth/login`	用户登录
POST	`/api/documents/upload`	上传文档
POST	`/api/documents/{id}/process`	处理文档
GET	`/api/documents/`	文档列表
POST	`/api/chat`	普通问答
POST	`/api/chat/stream`	SSE 流式问答
GET	`/api/conversations/`	会话列表
GET	`/api/conversations/{id}`	会话详情
GET	`/api/conversations/messages/{id}`	单条消息
GET	`/api/retrieval/debug/{id}`	检索调试
GET	`/api/health`	健康检查

三、系统架构与设计

3.1 整体架构图



3.2 Agent 交互流程

系统包含 6 个 Agent，各司其职：

Agent	职责	输入	输出
Planner	分析问题，拆解任务	User Query	Retrieval Plan
Query	Query Rewrite/Expansion	Original Query	Rewritten/Expanded Queries
Retriever	混合检索	Queries	Retrieved Documents
Rerank	重排序	Documents	Reranked Documents
Reflection	质量判断	Documents	Pass/Fail Decision
Answer	生成答案	Documents + Query	Answer + Citations

3.3 数据流设计

文档处理流程

文档上传 → 文档解析 → 文本切分 → Embedding 生成 → 存入 Milvus + BM25 索引

文档解析用了 PyMuPDF (PDF)、python-docx (DOCX)，文本和 Markdown 直接读取。切分用 RecursiveCharacterTextSplitter, chunk_size=500, overlap=100。Embedding 模型是 BGE Large Zh (bge-large-zh-v1.5)，1024 维。

问答流程

用户提问 → Planner 拆解 → Query 优化 → 混合检索 → Rerank → Reflection → Answer

混合检索把 Vector Search (Milvus) 和 BM25 的结果用 RRF 融合排序。Rerank 用的 BGE Reranker。Reflection 判断检索质量，不满意就重试，最多 2 次。

3.4 技术选型理由

组件	选型	理由
Agent 框架	LangGraph	原生支持状态机、条件分支、循环
向量数据库	Milvus	高性能、分布式、生产级
缓存	Redis	高性能、支持多种数据结构
Embedding	BGE Large Zh	中文效果最佳，MTEB 排名领先
Reranker	BGE Reranker	与 Embedding 配套，效果好
后端框架	FastAPI	异步、自动文档、类型安全
前端框架	Vue3	响应式、组合式 API、生态成熟

四、关键实现与代码展示

4.1 LangGraph Workflow 定义

```
# backend/src/workflows/rag_workflow.py
```

```
def build_rag_workflow() -> StateGraph:
    """Build the RAG workflow graph with all agents."""
    graph = StateGraph(RAGState)

    # Add agent nodes
    graph.add_node("planner", planner_agent)
    graph.add_node("query_agent", query_agent)
    graph.add_node("retriever", retriever_agent)
    graph.add_node("rerank", rerank_agent)
    graph.add_node("reflection", reflection_agent)
```

```

graph.add_node("answer", answer_agent)

# Define workflow edges
graph.set_entry_point("planner")
graph.add_edge("planner", "query_agent")
graph.add_edge("query_agent", "retriever")
graph.add_edge("retriever", "rerank")
graph.add_edge("rerank", "reflection")

# Conditional edge: retry or answer
graph.add_conditional_edges(
    "reflection",
    should_retry,
    {"answer": "answer", "query_agent": "query_agent"}
)

graph.add_edge("answer", END)
return graph.compile()

```

4.2 RAGState 状态定义

backend/src/agents/state.py

```

class RAGState(TypedDict):
    original_query: str
    rewritten_query: str
    expanded_queries: list[str]
    retrieval_plan: str
    retrieved_documents: list[RetrievalDocument]
    reranked_documents: list[RetrievalDocument]
    reflection_result: str
    reflection_passed: bool
    retry_count: int
    final_answer: str
    citations: list[CitationInfo]
    chat_history: Annotated[list, add_messages]
    conversation_id: int | None
    message_id: int | None
    workflow_trace: list[dict]

```

4.3 Planner Agent 实现

backend/src/agents/planner.py

```

async def planner_agent(state: RAGState) -> dict:
    """Analyze user query and generate retrieval plan."""
    query = state["original_query"]
    chat_history = state.get("chat_history", [])

    # Build conversation history context
    history_text = ""
    if chat_history:
        history_text = "\n".join([f"{msg.type}: {msg.content}" for msg in
chat_history[-6:]])

```

```
llm = get_llm()
prompt = f"""你是一个查询分析专家。分析用户问题并生成检索计划。
```

用户问题: {query}

```
{f'对话历史: {history_text}' if history_text else ''}
```

请返回 JSON 格式的检索计划:

```
{{
    "needs_decomposition": true/false,
    "sub_queries": ["子查询 1", "子查询 2"],
    "strategy": "描述检索策略",
    "key_entities": ["关键实体 1", "关键实体 2"]
}}
```

只返回 JSON, 不要其他内容。"""

```
try:
    response = await llm.ainvoke(prompt)
    plan_text = _extract_text(response.content)
    plan = json.loads(plan_text)
except Exception as e:
    logger.warning("planner_parse_failed", error=str(e))
    plan = {"needs_decomposition": False, "sub_queries": [query]}

return {
    "retrieval_plan": json.dumps(plan, ensure_ascii=False),
    "workflow_trace": state.get("workflow_trace", []) + [{"step": "planner",
"output": plan}],
}
```

4.4 混合检索实现

backend/src/retrieval/hybrid.py

```
async def hybrid_search(query: str, top_k: int = 10) -> list[dict]:
    """融合 Vector 和 BM25 检索"""
    # Check cache first
    cache_key = f"search:{hashlib.md5(query.encode()).hexdigest()}:{top_k}"
    try:
        redis = await get_redis()
        cached = await redis.get(cache_key)
        if cached:
            return json.loads(cached)
    except Exception:
        pass

    # Run BM25 and vector search in parallel
    bm25_index = get_bm25_index()
    bm25_task = asyncio.to_thread(bm25_index.search, query, top_k)
    vector_task = search_similar(query, top_k=top_k)
```

```

bm25_results, vector_results = await asyncio.gather(
    bm25_task, vector_task, return_exceptions=True,
)

# Merge by chunk_id
merged: dict[int, dict] = {}
for r in bm25_results:
    key = r["chunk_id"]
    merged[key] = {**r, "bm25_score": r["score"], "vector_score": 0.0}

for r in vector_results:
    key = r["chunk_id"]
    if key in merged:
        merged[key]["vector_score"] = r["score"]
    else:
        merged[key] = {**r, "bm25_score": 0.0, "vector_score": r["score"]}

# RRF ranking
ranked = rrf_rank(list(merged.values()), top_k=top_k)

# Cache results
try:
    redis = await get_redis()
    await redis.setex(cache_key, 300, json.dumps(ranked, ensure_ascii=False))
except Exception:
    pass

return ranked

def rrf_rank(results: list[dict], k: int = 60, top_k: int = 10) -> list[dict]:
    """Reciprocal Rank Fusion"""
    for r in results:
        bm25_rank = r.get("bm25_rank", k + 1)
        vector_rank = r.get("vector_rank", k + 1)
        r["rrf_score"] = 1.0 / (k + bm25_rank) + 1.0 / (k + vector_rank)

    return sorted(results, key=lambda x: x["rrf_score"], reverse=True)[:top_k]

```

4.5 SSE 流式输出实现

backend/src/api/v1/chat.py

```

@router.post("/stream")
async def chat_stream_endpoint(
    request: ChatRequest,
    current_user: User = Depends(get_current_user),
):
    """SSE streaming with real-time agent progress events."""

    async def event_generator():
        try:
            async for event_type, event_data in chat_stream(
                user_id=current_user.id,

```

```

        message=request.message,
        conversation_id=request.conversation_id,
    ):
        if event_type == "progress":
            yield {
                "event": "progress",
                "data": json.dumps(event_data, ensure_ascii=False),
            }
        elif event_type == "final":
            yield {
                "event": "token",
                "data": json.dumps({"text": event_data.get("answer", "")},
ensure_ascii=False),
            }
            yield {
                "event": "final",
                "data": json.dumps(event_data, ensure_ascii=False, default=str),
            }
        elif event_type == "error":
            yield {
                "event": "error",
                "data": json.dumps(event_data, ensure_ascii=False),
            }
    except Exception as e:
        yield {"event": "error", "data": json.dumps({"detail": str(e)})}

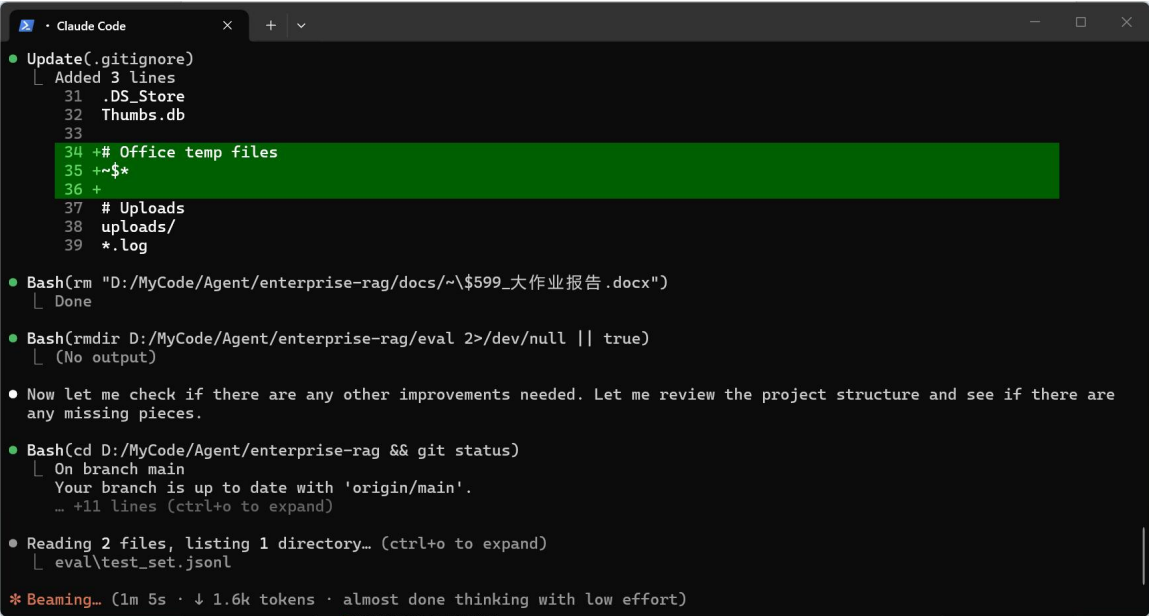
    return EventSourceResponse(event_generator())

```

4.6 AI IDE 辅助开发

开发过程中用了 Claude Code 作为辅助工具，从需求分析、架构设计到代码实现都有参与。

Claude Code 使用截图：



AI IDE 辅助开发的关键场景：

场景	AI IDE 贡献
架构设计	生成分层架构模板、Agent 接口定义
代码实现	生成 LangGraph Workflow、Agent 逻辑、API 路由
测试编写	生成单元测试、Mock 配置
调试优化	分析错误日志、建议性能优化方案
文档生成	生成 Specs 规格文档、API 文档

五、测试与评估

5.1 单元测试

项目包含完整的单元测试，覆盖以下模块：

测试模块	测试文件	测试内容
Agents	test_planner.py, test_reflection.py, test_state.py	Agent 逻辑、状态定义
API	test_auth.py, test_chat.py, test_documents.py, test_health.py	API 端点
Memory	test_conversation_memory.py	Redis 对话记忆
Retrieval	test_bm25.py, test_chunker.py, test_hybrid.py	检索管线
Schemas	test_chat.py, test_common.py, test_document.py	Pydantic 验证
Utils	test_file_utils.py, test_text_utils.py	工具函数
Workflows	test_document_pipeline.py, test_rag_workflow.py	工作流

5.2 功能测试

测试项	测试内容	结果
文档上传	PDF/DOCX/TXT/Markdown	☒ 通过
文档处理	解析、切分、Embedding	☒ 通过
混合检索	Vector + BM25 融合	☒ 通过
Query Rewrite	问题改写和扩展	☒ 通过
Rerank	BGE Reranker 重排序	☒ 通过
Reflection	检索质量判断	☒ 通过
多轮对话	上下文关联	☒ 通过
SSE 流式	实时 Agent 进度推送	☒ 通过
引用来源	Citation 展示和跳转	☒ 通过

5.3 性能指标

以下数据基于本地开发环境手动测试（Intel i5 / 16GB RAM / 无 GPU）：

指标	测试条件	结果	说明
单次问答响应时间	含 3 次 LLM 调用	~60-70s	主要瓶颈为 LLM

			API 延迟（15-25s/次）
文档处理速度	PDF 解析 + 切分 + Embedding	~100 页/min	Embedding 为 CPU 推理
混合检索延迟	BM25 + Vector + RRF 融合	< 500ms	Redis 缓存命中时 < 50ms
Rerank 延迟	BGE Reranker（首次加载）	~10s	模型加载后单次 < 200ms

优化措施：

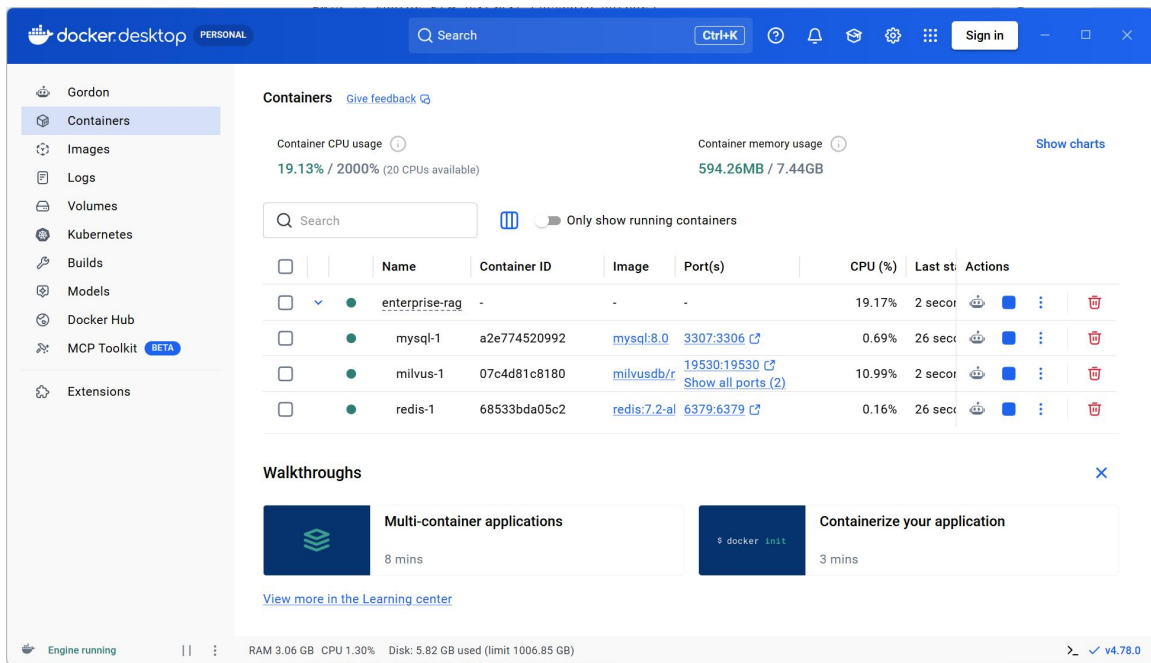
LLM 客户端缓存复用（避免重复创建实例）

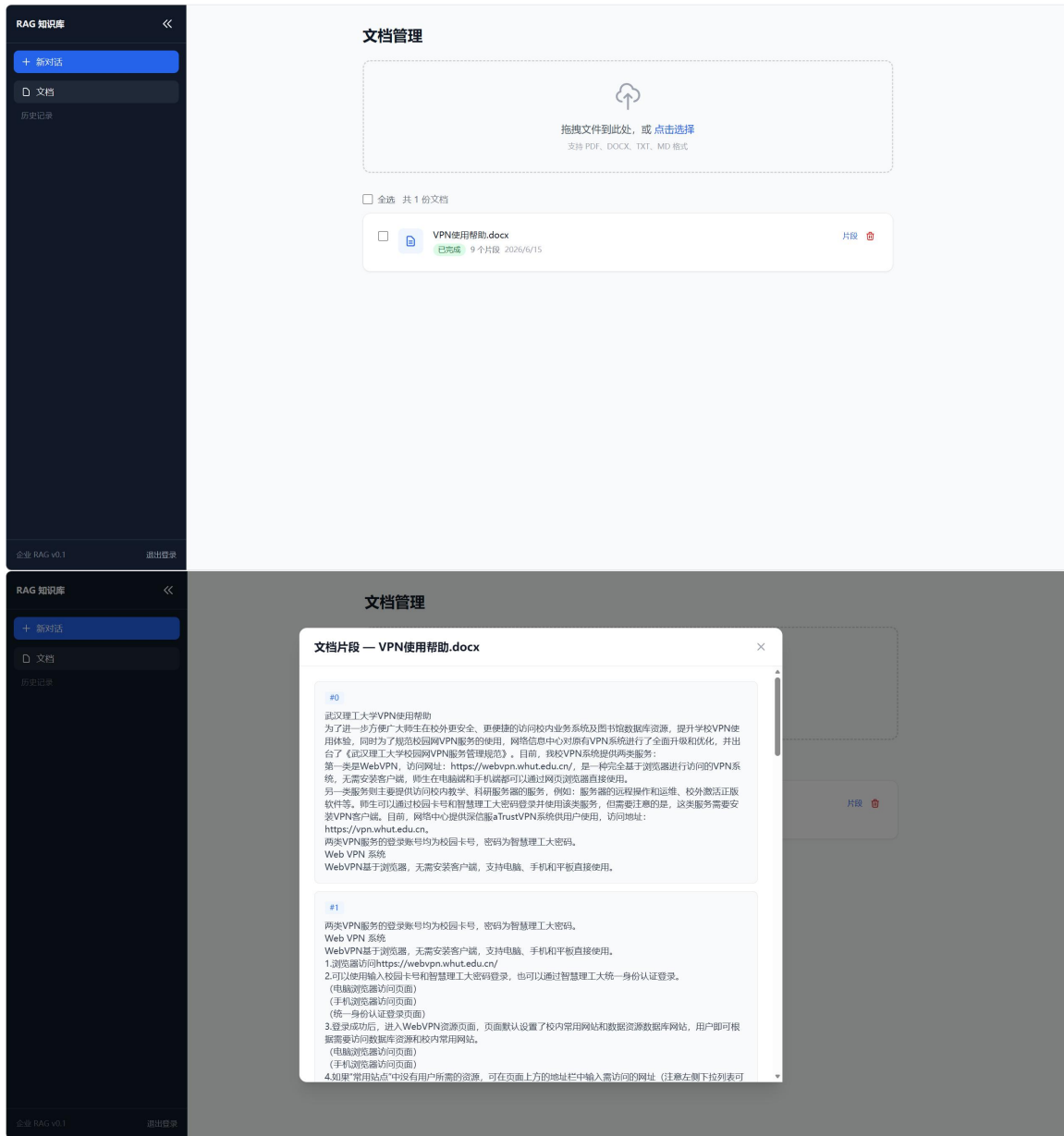
BM25 与 Vector 检索并行执行（`asyncio.gather`）

检索结果 Redis 缓存（5min TTL）

Reranker 模型预热（启动时加载）

5.4 Demo 截图





六、系统升级与扩展

6.1 可扩展架构

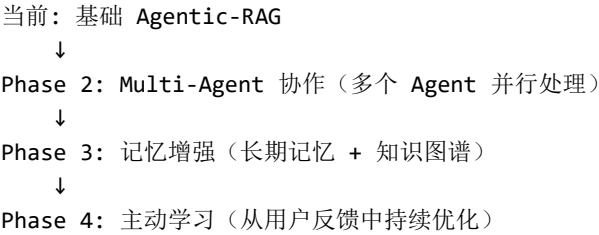
系统采用分层架构和依赖注入，扩展起来比较方便。新增 Agent 只要实现接口加到工作流里就行；检索管线也能加新的策略，比如 Knowledge Graph；存储层支持水平扩展，Milvus 和 Redis 都能搞集群。

6.2 下一阶段计划

阶段	功能	技术方案
----	----	------

Phase 2	权限管理系统	RBAC + JWT
Phase 2	文档版本管理	版本号 + 差异对比
Phase 3	多租户支持	数据隔离 + 资源配额
Phase 3	知识图谱增强	Neo4j + GraphRAG

6.3 能力演进路径



七、课程总结

7.1 个人收获

做这个项目最大的收获是思维方式的转变。以前写代码就是写函数调用，现在得想怎么设计 Agent workflow，怎么让各个 Agent 配合起来。SDD 方法论也实践了一下，先写 Spec 再写代码，确实能提高效率。系统设计方面，分层架构和依赖注入的好处在实际开发中体会很深，改一个模块不会影响其他的。另外就是 AI 应用从原型到生产的链路，错误处理、性能优化、可观测性这些以前没怎么关注的东西，这次都踩了一遍坑。

7.2 对课程的建议

增加 Agent 评估的实践指导，怎么科学地评估 Agent 系统的效果

提供更多企业级案例参考，帮助理解实际业务场景

加强 LangGraph 等框架的深入讲解，特别是状态管理和条件分支

7.3 总结

这个项目从零搭了一个企业级 Agentic-RAG 智能知识库系统，实现了多阶段检索、自适应优化和精准引用。通过这个项目，对 Agentic AI 的设计理念和工程实践有了比较深入的理解，为以后做 AI 应用开发打下了基础。

附录

A. 项目仓库

GitHub: <https://github.com/wseacat/cs599-project.git>

B. 环境变量配置

```
# .env.example
APP_ENV=development
SECRET_KEY=your-secret-key
LLM_API_KEY=your-api-key
LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
LLM_MODEL=qwen-plus
DATABASE_URL=mysql+aiomysql://root:changeme@localhost:3306/enterprise_rag
REDIS_HOST=localhost
REDIS_PORT=6379
MILVUS_HOST=localhost
MILVUS_PORT=19530
```

C. 快速启动

1. 复制环境配置

```
cp .env.example .env
```

2. 启动基础设施

```
docker compose up -d
```

3. 启动后端

```
cd backend
```

```
pip install -e .
```

```
uvicorn src.main:app --reload --port 8000
```

4. 启动前端

```
cd frontend
```

```
npm install
```

```
npm run dev
```